1.
Tienes un editor gráfico con múltiples herramientas (pincel, borrador, línea, relleno). El usuario puede cambiar de herramienta sin que la clase principal del editor tenga que saber **cómo funciona cada herramienta internamente**.

¿Cuál es el patrón más adecuado para estructurar este diseño?
a) **State**
b) **Command**
c) **Strategy**
d) **Observer**

2.
En una aplicación bancaria, una transacción puede ser **ejecutada, deshecha o rehecha**. Además, quieres almacenar un historial de operaciones para poder aplicar un "deshacer múltiple" fácilmente.

¿Qué patrón se ajusta mejor a este escenario?
a) **Strategy**
b) **Command**
c) **State**
d) **Observer**

3.
En un sistema de domótica, múltiples sensores (temperatura, humedad, movimiento…) deben **notificar cambios** a diferentes módulos (por ejemplo, climatización, seguridad, registros).
No quieres acoplar directamente cada sensor a cada módulo.

¿Qué patrón aplicarías?
a) **Observer**
b) **Mediator**
c) **Chain of Responsibility**
d) **Strategy**

4.
Un formulario web complejo tiene múltiples componentes (campos, botones, menús desplegables) que deben **reaccionar entre sí** cuando cambia uno de ellos, pero no quieres que los componentes estén directamente acoplados entre sí.

¿Qué patrón utilizarías para **centralizar la lógica de interacción**?
a) **Observer**

b) **Mediator**
c) **State**
d) **Command**

5.
En una aplicación de validación de documentos, un archivo debe pasar por varias etapas opcionales (validación de formato, validación de firma digital, validación semántica, etc.).
Cada etapa **puede decidir si procesa el documento o lo delega a la siguiente etapa**.
La secuencia de validación debe poder **extenderse fácilmente** sin modificar las validaciones existentes.

¿Qué patrón usarías?
a) **Strategy**
b) **Chain of Responsibility**
c) **Mediator**
d) **State**